

PA 1: Implementing xv6 User Commands and System Calls

Task 1. (5 pts) Implement the uptime utility

```
$ uptime  
up 47 clock ticks
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

```
xv6 kernel is booting  
  
init: starting sh  
$ time1 sleep 10  
elapsed time: 10 ticks  
$ time1 matmul  
Time: 14 ticks  
elapsed time: 14 ticks  
$ time1 matmul & time1 matmul &  
$ Time: 24 ticks  
Time: 25 ticks  
elapsed time: 25 ticks  
elapsed time: 26 ticks
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

First, I modified proc.h file to declare cputime variable within the 'struct proc' using 'uint cputime'. Next is to modify proc.c to initialize the variable to 0 in the allocproc() struct using 'p->cputime = 0'. Lastly, I modified trap.c in adding the functionality of incrementing the time such as 'p->cputime++'.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage. Added a cputime field to struct proc. This gives every process its own running counter of how many timer-interrupt ticks it's actually been scheduled to run on the CPU, as opposed to how much wall-clock time has simply passed.

proc.c In allocproc(), initialize cputime = 0 for every newly created process, so it starts counting from zero. Added a new function kwait2(), duplicated from the existing kwait(), which does everything kwait() already did and copy that child's accumulated cputime

trap.c In usertrap() and kerneltrap(), added cputime++ at the point where a process's timer slice expires and it's about to yield the CPU.

pstat.h Defines struct rusage { uint cputime; }

sysproc.c Added sys_wait2(), duplicated from sys_wait().

defs.h Added the prototype for kwait2() so that sysproc.c (a different file) is allowed to call a function defined in proc.c.

syscall.h Added #define SYS_wait2 23 reserving a syscall ID number for wait2

syscall.c Added the extern uint64 sys_wait2(void); declaration, and the [SYS_wait2] = sys_wait2, entry in the syscalls[] dispatch array.

user.h Added struct rusage; and the declaration int wait2(int*, struct rusage*).

usys.pl Added entry("wait2");

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

```
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul &
$ Time: 24 ticks
Time: 23 ticks
elapsed time: 25 ticks, cpu time: 12 ticks, 48% CPU
elapsed time: 25 ticks, cpu time: 13 ticks, 52% CPU
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

I learned how to implement system calls to track elapsed time and CPU time and how using certain commands in chain can increase the time. I did struggle in getting qemu to run which is weird because when I finished Tasks 2 the first time it worked perfectly fine. I had to use AI to make it runnable but hopefully I do not have to do that again because I still want to be able to deduce some of the more complex errors lines myself.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

Our time command only measures the CPU time of the single process it directly forks and executes. If that process spawns its own child processes internally, any CPU time those descendants consume is invisible to time wait2().